Deep learning

The Perceptron

Course materials

The github https://github.com/SkatAI/deeplearning

- slides
- notebooks
- code
- PDF documents

Google Colab: https://colab.research.google.com/

- offers some GPU

Perceptron
Rosenblatt 1957

Perceptron - 1957

April 11, 1957, Queen Elizabeth II visits Paris
©Getty - Express

Some dates

1943: McCulloch and Pitts propose the notion of an artificial neuron combining inputs to produce an output, without a practical learning algorithm.

1957: Rosenblatt develops the **perceptron**, which linearly

- combines inputs and thresholds to make a binary decision,
- with an algorithm for learning weights from the data.

1969: Minsky and Papert: multilayer perceptron

1980s: Development of backpropagation

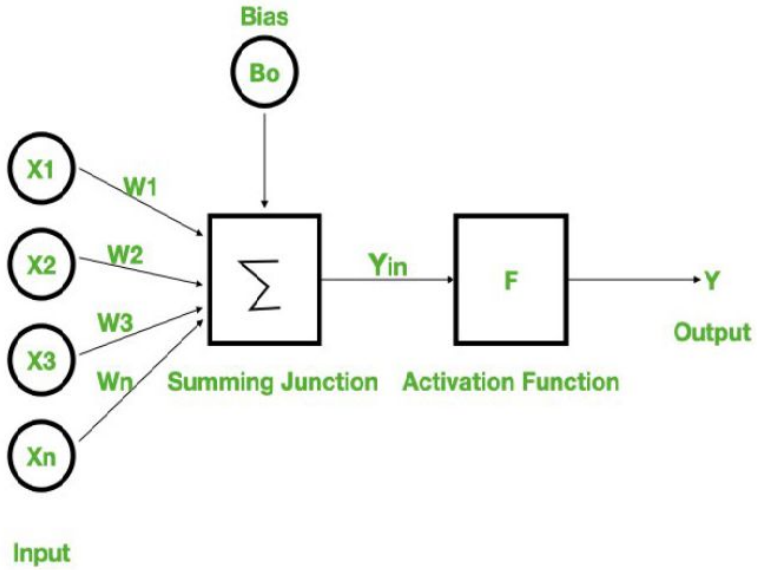
https://en.wikipedia.org/wiki/Neural_network_(machine_learning)

Basically, the neuron

- coefficients - weights + a bias
  - Equivalent to a linear regression

and an **activation function**

- This is commonly a step function that outputs:
  - 1 (or active/fire) if the weighted sum exceeds the threshold
  - 0 (or inactive/don't fire) if it doesn't
- The activation function introduces non linearity



https://www.geeksforgeeks.org/how-neural-networks-can-be-used-for-data-mining/

Presentation of the perceptron

linearly separable data

- N samples
- parameters: coefficients, learning rate, classification threshold

- initialization: coefficients = 0 (N)
- for each randomly chosen sample:
  - `value = dot_product(sample, coefficients) + bias`

Regression

Heavyside Function

  - prediction = 1 if `value` > **threshold** otherwise prediction = 0

if prediction != true value => update coefficients:

Update

  - `coefs = coefs + learning_rate (true - prediction) * samples`

otherwise => nothing

Algorithm: Perceptron Learning Algorithm

$P \leftarrow$ *inputs with label* 1;
$N \leftarrow$ *inputs with label* 0;
Initialize $\mathbf{w}$ randomly;
while !*convergence* **do**
    Pick random $\mathbf{x} \in P \cup N$;
    if $\mathbf{x} \in P$ *and* $\mathbf{w}.\mathbf{x} < 0$ **then**
        $\mathbf{w} = \mathbf{w} + \mathbf{x}$;
    end
    if $\mathbf{x} \in N$ *and* $\mathbf{w}.\mathbf{x} \geq 0$ **then**
        $\mathbf{w} = \mathbf{w} - \mathbf{x}$;
    end
end
//the algorithm converges when all the inputs are classified correctly

Single layer perceptron ~= logistic regression

in both cases: linear combination of input values

Few differences:

	single-layer perceptron	**logistic regression**
activation function	heavyside	sigmoid (logit)
training	stochastic gradient	MLE / maximum likelihood
interpretation	-	Statistical

Single layer perceptron vs Stochastic gradient descent

The perceptron algorithm is a special case of SGD

- specific error function
- a unit step activation function.

	Perceptron	Stochastic Gradient Descent (SGD)
Definition	Specific learning algorithm for single-layer neural networks	General optimization algorithm applicable to many ML models
Learning Process	Iterative, processes one example at a time	Iterative, processes one or mini-batches of examples
Update Trigger	Updates only when misclassification occurs	Updates based on gradient of loss function on every example
Error Function	Discrete error (0 if correct, 1 if incorrect)	Continuous loss functions (MSE, cross-entropy, etc.)
Convergence	Guaranteed only for linearly separable data	Can converge on non-linearly separable problems with appropriate models
Learning Rate	Traditionally uses fixed learning rate	Often employs adaptive or scheduled learning rates
Primary Use	Binary classification	Optimization across various ML models including deep networks
Weight Updates	Moves weights in direction that reduces error	Moves weights in direction of negative gradient of loss function

Limit of the perceptron

A single-layer perceptron cannot learn nonlinear functions, despite the fact that the activation function (Heaviside function) is nonlinear.

The perceptron does not have hidden layers which are needed to capture nonlinear relationships between inputs and output.

To learn nonlinear functions, more advanced neural network architectures are used. For example, multi-layer perceptrons with hidden layers associated with nonlinear activation functions.

XOR vs OR

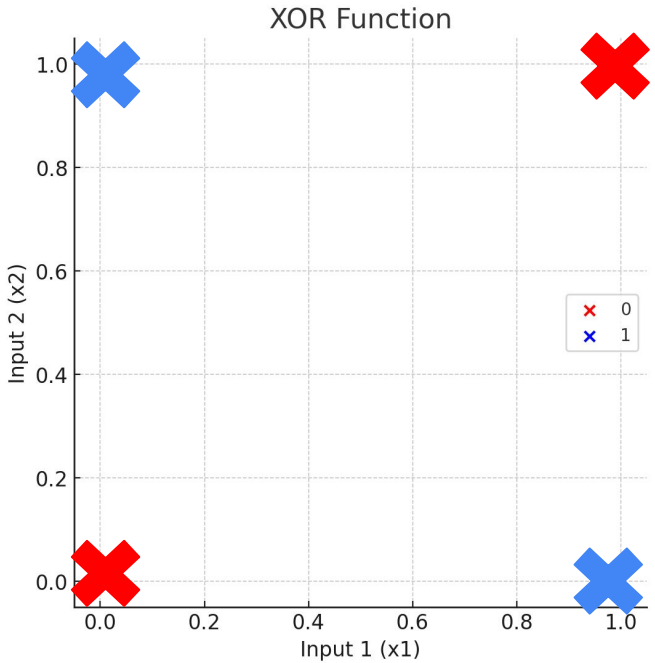
A single-layer perceptron can not learn XOR

OR

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

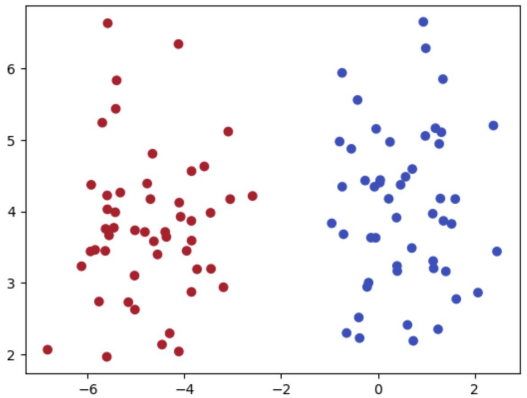
XOR

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0



notebook perceptron

- perceptron code
  - basic version
  - then add the learning rate
- Demo OR, XOR
- influence of weight initialization and bias
- generate blobs
  - separable and non-separable
- experiment with
  - learning rate
  - epochs



https://colab.research.google.com/drive/1hHPK0y0NvSOLJDrXHtEIoiz4kMr1jqub#scrollTo=Uzq6vJDAJqZC

Questions ?

questions

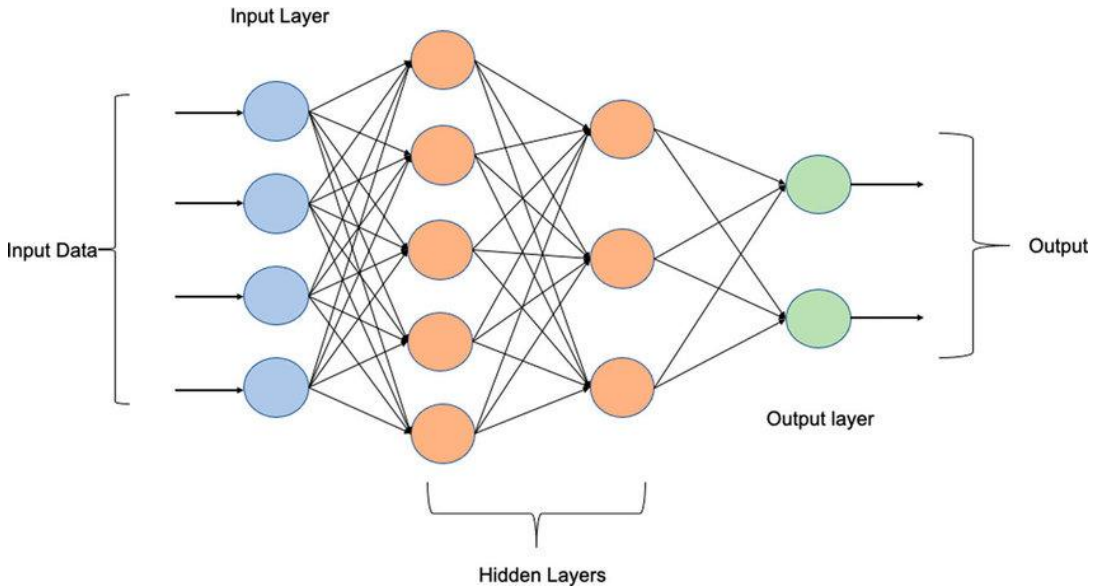
Why introduce bias?

- additional degree of freedom that allows the translation of the decision curve
- if the sample has only values = 0 => all outputs = 0
- changes the activation level of the perceptron

Multi-Layer Perceptron (MLP)

MLP

multi-layer perceptron = perceptron + hidden layers (inner layers)



activation functions

sigmoid [0,1]

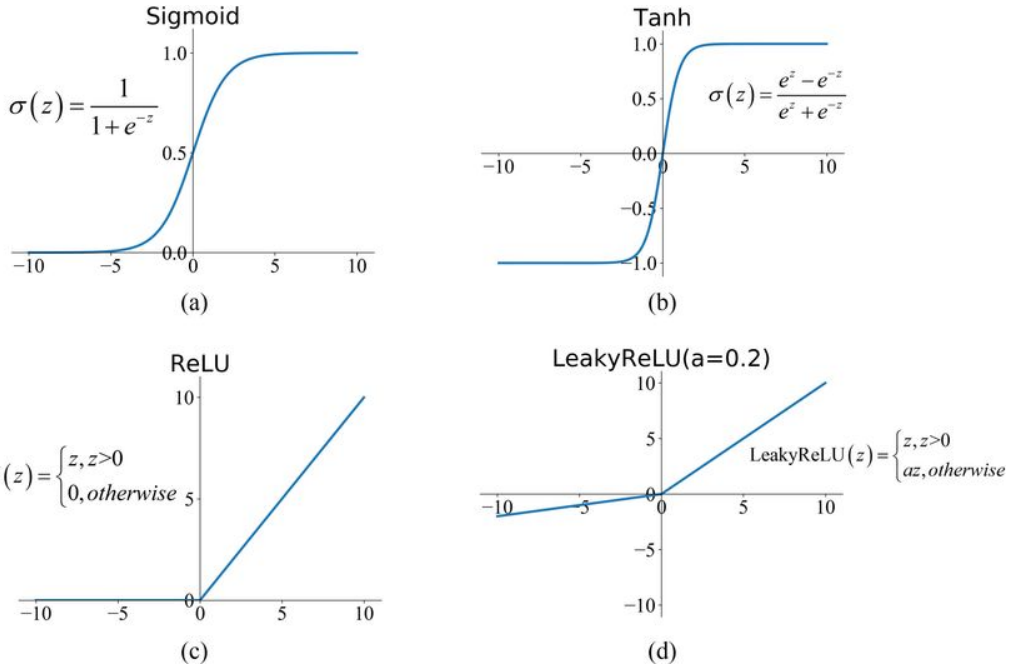
- gradient vanishing, calculations, not centered at 0 => slows down convergence

tanh: [0,1]

- expensive calculation (exp)

ReLu [0, inf]

- gradient = 0 or 1
- accelerated convergence
- preferred choice



activation functions

- Introduce non-linearity, allowing networks to learn complex patterns and relationships that linear models cannot capture
- Control the neuron's output signal strength, determining whether and how strongly a neuron "fires"
- Enable gradient-based learning by providing derivatives for backpropagation (except for step functions)

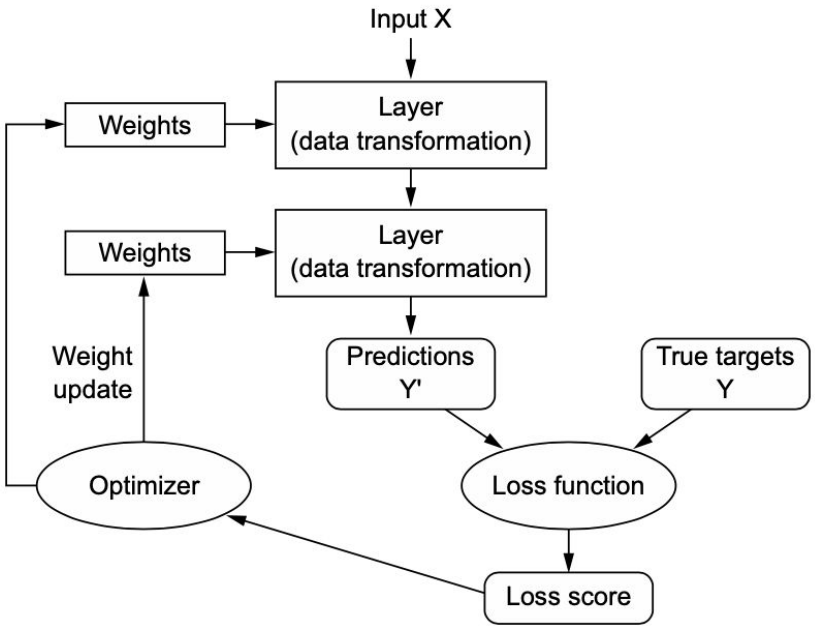
And

- Help mitigate issues like vanishing/exploding gradients when properly selected
- Transform unbounded inputs into bounded outputs when using functions like sigmoid or tanh
- Enable specialized behavior in different network architectures (e.g., softmax in classification output layers)

Optimizer - solver

SGD vs ADAM vs RMSprop

backpropagation algorithm which updates the neuron coefficients based on the gradient of the cost function

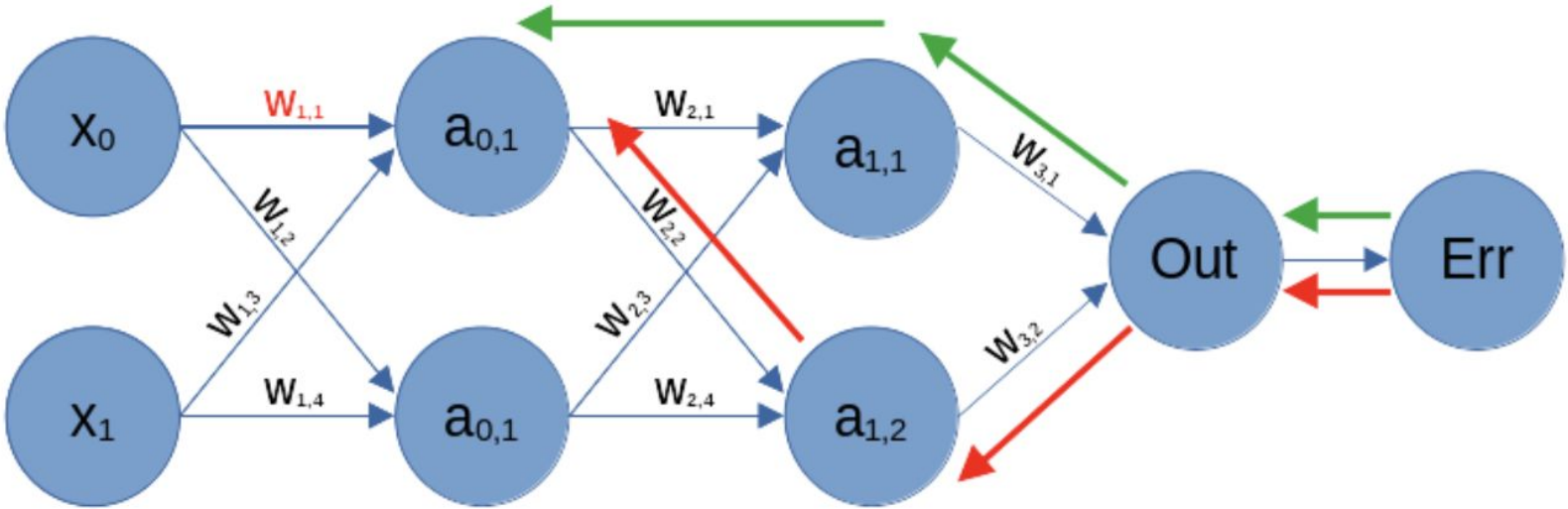


Backpropagation

Backpropagation is the algorithm that enables neural networks to learn by:

- Calculating how much each weight contributes to the overall error
- Efficiently computing gradients by applying the chain rule of calculus
- Propagating error signals backward through the network
- Enabling weight updates that minimize the loss function

Backpropagation allows neural networks to adjust their parameters based on their mistakes.
It makes learning through gradient descent possible.



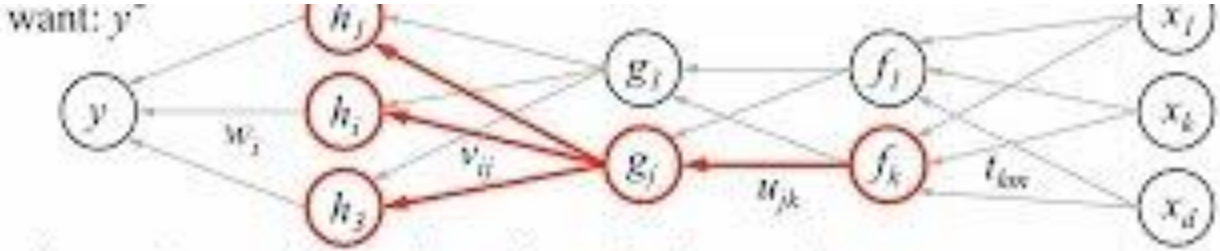
Lifecycle of a node's value

Lifecycle of a node's value:

- initialization has values, (Xavier or He strategy)
- **forward pass:**
  - from start to finish (input to output, left to right)
  - for each node, calculation of the weighted average with the node coefficients => calculation of the output
  - output values: stored for backward pass
- calculation of the cost function value, of the error at the output of the graph
- **backward pass** :
  - gradient calculation of the cost function
  - propagation from end to beginning
  - update the coefficients of each node with the gradient descent
- settings update

For inference, the node values are fixed and the network can make predictions with the forward pass

Backpropagation



1. receive new observation $x = [x_1 \ldots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer 1…L
   compute g_j based on units f_k from previous layer: $g_j = \sigma\left(u_{j0} + \sum_k u_{jk} f_k\right)$
3. get prediction y and error $(y-y^*)$
4. **back-propagate error:** for each unit g_j in each layer L…1

(a) compute error on g_j

$$\frac{\partial E}{\partial g_j} = \sum_i \sigma'(h_i) v_{ij} \frac{\partial E}{\partial h_i}$$

$\frac{\partial E}{\partial g_j}$: should g_j — $\sigma'(h_i) v_{ij}$: how h_i will — $\frac{\partial E}{\partial h_i}$: was h_i too

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

$$\frac{\partial E}{\partial u_{jk}} = \frac{\partial E}{\partial g_j} \sigma'(g_j) f_k$$

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

MLP

- can learn non-linear functions using activation function and hidden layers

but

- possible existence of several local minima => sensitive to initialization values
- it is necessary to optimize the hyper-parameters: # layers, # nodes, # epochs, # the learning rate
- sensitive to the dynamics of variables => it is necessary **to normalize**

MLP with scikit-learn

MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100,),
activation='relu', *, solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200,
shuffle=True, random_state=None, tol=0.0001, verbose=False, warm_start=False,
momentum=0.9, nesterovs_momentum=True, early_stopping=False,
validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08,
n_iter_no_change=10, max_fun=15000)                                    [source]
```

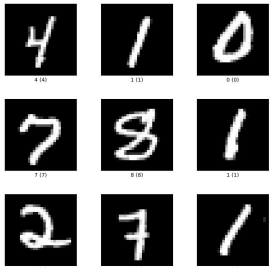
Multi-layer Perceptron classifier.

This model optimizes the log-loss function using LBFGS or stochastic gradient descent.

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

MLP workshop: MNIST with scikit-learn MLPClassifier

- implementation with scikit-learn on mnist
- MNIST
  - it's the Iris / Titanic of deep learning
  - 28x28 images - numbers 0 to 9
  - 60k train, 10k test
  - https://www.tensorflow.org/datasets/catalog/mnist
  - original 1998 http://yann.lecun.com/exdb/mnist/



- hyperparameter tuning
  - number of layers, number of nodes
  - binary or multiclass version vs regression

visualize the internal coefficients

see
https://scikit-learn.org/stable/auto_examples/neural_networks/plot_mnist_filters.html#sphx-glr-auto-examples-neural-networks-plot-mnist-filters-py